МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНОМУ УНІВЕРСИТЕТІ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”

Кафедра систем штучного інтелекту

Лабораторна робота №4

з дисципліни
«Операційні системи»

Виконав:
студент групи КН-114
Добрій Назарій

Викладач:
Кривенчук Ю.П.

Львів – 2020 р.

Тема. Синхронізація потоків в ОС Windows

Мета. Ознайомитися зі способами синхронізації потоків. Навчитися організовувати багатопоточність з використанням синхронізації за допомогою функцій WinAPI.

Завдання.

1. Реалізувати алгоритм із лабораторної роботи №3.
2. Здійснити розпаралелювання даного алгоритму на 2, 4, 8 потоків із використанням синхронізації.
3. Реалізувати прогрес (хід) виконання задачі.
4. Для синхронізації потоків використати такі методи: Interlocked-функції, м'ютекси, семафори, критична секція, спін-блокування, Wait-функції.
5. Результати виконання роботи відобразити у звіті.

Індивідуальні завдання.

Відповідно до вказаного варіанту реалізувати два механізми синхронізації:

Варіант № 1-12 – м'ютекс
Варіант № 13-25 – семафор

Варіант № 1-6 – Interlocked-функції
Варіант № 7-12 – критична секція
Варіант № 13-18 – спін-блокування
Варіант № 19-25 – Wait-функції

Варіант 6 – м'ютекс та Interlocked-функції

Код реалізацій програми:

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <windows.h>
#include <algorithm>
#include <vector>
#include <ctime>

using namespace std;

HANDLE hThreads[8];
HANDLE hThreadOpen[8];
HANDLE hMutex;

vector <int> ThreadFuncTakeNums;

volatile LONG TotalThreadCount = 0;

DWORD WINAPI selectionSort(void* param) {
    vector<int>& temp = *(vector<int>*) param;
    int min;

    WaitForSingleObject(hMutex, INFINITE);
    for (int i = 0; i < temp.size(); i++) {
        ThreadFuncTakeNums.push_back(temp[i]);
    }

    for (int i = 0; i < temp.size() - 1; i++) {

        min = i;
        for (int j = i + 1; j < temp.size(); j++)
            if (temp[j] < temp[min])
```

```
            if (temp[j] < temp[min])
                min = j;
        swap(temp[i], temp[min]);

    }
    ReleaseMutex(hMutex);

    return 0;
}

vector<int> complete(vector<vector<int>> arr, int n, int k) {
    vector<int> completed(n);
    for (int i = 0; i < n; i++) {
        int min = 1000000;
        int th = -1;
        for (int j = 0; j < k; j++) {
            if (!arr[j].empty() && arr[j][0] < min) {
                min = arr[j][0];
                th = j;
            }
        }
        completed[n - i - 1] = min;
        arr[th].erase(arr[th].begin());
    }
    return completed;
}

int main() {
    srand(time(NULL));

    DWORD dwIncre;
```


```
    DWORD dwIncre;
```


```
    //creating mutex
    hMutex = CreateMutex(
        NULL,
        FALSE,
        NULL
    );

    int elem_num, thread_num;
    vector<vector<int>> arr;

    cout << "Enter the number of elements: ";
    cin >> elem_num;
    vector<int> input(elem_num);
    for (int i = 0; i < elem_num; ++i) {
        input[i] = rand() % elem_num + 1;
        cout << input[i] << " ";
    }
    cout << endl;

    cout << "Enter the number of threads: ";
    cin >> thread_num;
    for (int i = 0; i < thread_num; ++i) {
        vector<int> tmp;
        for (int j = i * elem_num / thread_num; j < (i + 1) * elem_num / thread_num; j++) {
            tmp.push_back(input[j]);
        }
        arr.push_back(tmp);
    }

    int last_interation = 0;

    for (int i = 0; i < thread_num; i++) {
        int ThreadPriority;

        DWORD ThreadID;

        HANDLE thread = CreateThread(NULL, 0, selectionSort, &arr[i], 0, &ThreadID);
        hThreads[i] = thread;

        cout << "Number of thread - " << i + 1 << endl;

        //ThreadID
        cout << "Thread ID = " << ThreadID << endl;

        cout << "Values, which take this " << ThreadID << " thread:" << endl;

        for (int k = last_interation; k < ThreadFuncTakeNums.size(); k++)
        {
            cout << ThreadFuncTakeNums[k] << " ";
        }
        cout << endl;

        if (hThreads[i] == NULL)
        {
            cout << "Thread Error" << endl;
            ExitProcess(1);
        }

        //Opend Thread
```

```
        //Opend Thread
        HANDLE hOpenThread = OpenThread(
            THREAD_SET_LIMITED_INFORMATION,
            FALSE,
            ThreadID
        );

        //interlock function
        dwIncre = InterlockedIncrement((LPLONG)&TotalThreadCount);

        hThreadOpen[i] = hOpenThread;

        if (NULL == hOpenThread)
        {
            cout << "Open Thread Function Failed & Error - " << GetLastError() << endl;
            ExitProcess(2);
        }

        // Set Thread Priority
        int mode;
        cout << "Set priority: " << endl;
        cout << "Enter index of priority Idle(1), Below Normal(2), Normal(3), Above Normal(4), High(5), Realtime(6) = ";
        cin >> mode;

        while (mode > 7 || mode < 0)
        {
            cin >> mode;
        }

        switch (mode) {
        case 1:
```

```
        case 1:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_LOWEST);
            break;
        case 2:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_BELOW_NORMAL);
            break;
        case 3:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_NORMAL);
            break;
        case 4:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_ABOVE_NORMAL);
            break;
        case 5:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_HIGHEST);
            break;
        case 6:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_TIME_CRITICAL);
            break;
        }

        //Getting Thread Priority
        ThreadPriority = GetThreadPriority(thread);

        cout << "Thread Priority = " << ThreadPriority << endl;

        //Closing Handle of Thread
        CloseHandle(thread);

        cout << "Interlock increment = " << dwIncre << endl;

        last_interation += ThreadFuncTakeNums.size();
```

```

        //Closing Handle of Thread
        CloseHandle(thread);

        cout << "Interlock increment = " << dwIncre << endl;

        last_interation += ThreadFuncTakeNums.size();
        cout << endl;
    }

    for (int i = 0; i < thread_num; ++i) {
        //WaitForSingleObject for Thread
        WaitForSingleObject(hThreadOpen[i], INFINITE);
        CloseHandle(hThreadOpen[i]);
    }

    vector<int> completed = complete(arr, elem_num, thread_num);
    cout << "Sorting number:" << endl;
    for (int i = 0; i < elem_num; ++i) {
        cout << completed[i] << " ";
    }
    cout << endl;

    system("pause");
    return 0;
}
```

Скріни реалізацій завдання (console):

```
C:\Users\User\source\repos\712\Debug\712.exe
Enter the number of elements: 100

Enter the number of threads: 3
Number of thread - 1
Thread ID = 5592
Values, which take this 5592 thread:
Set priority:
Enter index of priority Idle(1), Below Normal(2), Normal(3), Above Normal(4), High(5), Realtime(6) = 3
Thread Priority = 0
Interlock increment = 1

Number of thread - 2
Thread ID = 12056
Values, which take this 12056 thread:
Set priority:
Enter index of priority Idle(1), Below Normal(2), Normal(3), Above Normal(4), High(5), Realtime(6) = 3
Thread Priority = 0
Interlock increment = 2

Number of thread - 3
Thread ID = 8620
Values, which take this 8620 thread:
Set priority:
Enter index of priority Idle(1), Below Normal(2), Normal(3), Above Normal(4), High(5), Realtime(6) = 3
Thread Priority = 0
Interlock increment = 3

Sorting number:
99 98 97 94 93 92 92 92 91 91 89 89 89 87 86 85 85 84 83 81 81 80 78 77 76 72 72 70 69 69 69 67 65 63 63 63 62 57 56
56 54 54 54 53 52 52 52 51 50 49 48 48 47 45 44 43 42 41 41 38 36 36 35 34 33 33 33 32 30 30 30 29 28 27 25 21 20 19
19 18 18 17 17 15 13 12 12 11 10 10 10 9 9 9 7 5 3 3 1 1
Press any key to continue . . .
```

Висновок: я ознайомитися зі способами синхронізації потоків такими як м’ютекс та Interlock функції. Штучно закривав доступ іншим потокам до однієї функції, отже один потік міг працювати з однією функцією.